

MINI ASSESSMENT

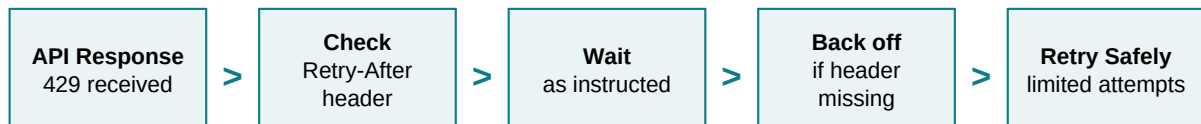
API Error Handling and Safe JSON Access

Name	DUDIGAM DEEPIKA ANJANI VENKATA LAKSHMI
Reg. No.	99230040297
Department	CSE - DS (Data Science)

Question 1

If an API returns 429, what should your program do?

FLOW OF HANDLING A 429 RESPONSE



Answer

HTTP status 429 means **Too Many Requests**: the client has crossed the API's rate limit. The program should stop sending requests immediately and wait before trying again. First, it should look for a **Retry-After** header and wait for exactly that long. If the header is not present, it should use **exponential backoff**, doubling the delay after each failure (1s, 2s, 4s, 8s). Retries must be limited to a fixed maximum so the program never loops forever. After the limit is reached, the error should be logged and handled gracefully instead of crashing.

```
import time, requests

def get_with_retry(url, max_retries=4):
    delay = 1
    for attempt in range(max_retries):
        r = requests.get(url)
        if r.status_code != 429:
            return r
        wait = int(r.headers.get("Retry-After", delay))
        time.sleep(wait)
        delay *= 2
    return None # give up after max_retries
```

Question 2

If response.json() fails, what should your program do?

FLOW OF HANDLING A JSON PARSE FAILURE



Answer

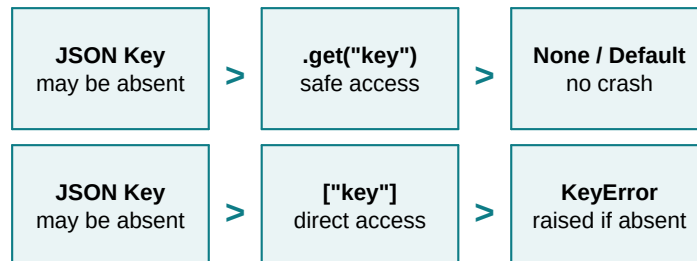
The call to **response.json()** should be placed inside a **try / except** block so that an empty or malformed body does not crash the program. The code should catch the decoding error (for example *ValueError* or *json.JSONDecodeError*). It is good practice to check the HTTP status code first, because an error status (404, 500 and so on) often returns HTML or plain text instead of JSON. Printing or logging *response.text* helps in debugging. Finally, the program should show a clear error message and either use a safe default value or stop only that operation, rather than letting the whole application fail.

```
try:
    data = response.json()
except ValueError:
    # includes JSONDecodeError
    print("Invalid JSON. Status:", response.status_code)
    print("Body:", response.text[:200])
    data = {}
    # safe fallback
```

Question 3

If a JSON key is missing, why might `.get()` be safer than `[]`?

SAFE VS. UNSAFE KEY ACCESS



Answer

`data.get("key")` is safer because it returns **`None`** (or a default value that we choose) when the key is missing, so the program keeps running. In contrast, **`data["key"]`** assumes the key always exists and raises a **`KeyError`** if it does not, which crashes the program unless it is handled separately. API responses often leave out optional fields, so **`.get()`** is the better choice for uncertain data.

```
user = {"name": "Deepika"}
print(user.get("email", "not provided")) # not provided
print(user["email"])                     # KeyError
```

QUICK COMPARISON

Method	Key Exists	Key Missing
<code>data.get("key")</code>	Returns value	Returns <code>None</code> / default
<code>data["key"]</code>	Returns value	Raises <code>KeyError</code>

KEY TAKEAWAY: Reliable API programs should handle rate limits, invalid responses, and missing JSON fields explicitly. This prevents unexpected crashes and makes the application resilient to real-world API behaviour.